In [2]:

```
import pandas as pd
import numpy as np
import re

# ==== 1. IMPORT DỮ LIỆU ====
# Đổi đường dẫn dưới đây thành file CSV của bạn
INPUT_PATH = "/kaggle/input/datasets/holthin/credit/D3-CreditScore.csv"
OUTPUT_PATH = "D3-CreditScore-cleaned.csv"

df = pd.read_csv(INPUT_PATH, low_memory=False)
print("Đã đọc file:", INPUT_PATH)
print("Shape ban đầu:", df.shape)
print(df.head())

orig_shape = df.shape

# --- helper: strip garbage wrapper characters like __10000__, 8_, trailing/
def clean_numeric_str(s):
    if pd.isna(s):
        return np.nan
    s = str(s).strip()
    s = s.replace('_', '')  # remove stray underscores (28_, __10000__, 8_)
    s = s.replace(',', '')
    if s in ('', '-', 'nan', 'NA', 'NaN', '!@9#%8'):
        return np.nan
    try:
        return float(s)
    except ValueError:
        return np.nan

numeric_str_cols = ['Age', 'Annual_Income', 'Num_of_Loan', 'Num_of_Delayed_P
                    'Changed_Credit_Limit', 'Outstanding_Debt', 'Amount_inv
                    'Monthly_Balance']

for c in numeric_str_cols:
    df[c] = df[c].apply(clean_numeric_str)

# Age: remove impossible values (negative, absurdly large)
df.loc[(df['Age'] < 0) | (df['Age'] > 100), 'Age'] = np.nan

# Num_of_Loan: negative codes like -100 are invalid
df.loc[df['Num_of_Loan'] < 0, 'Num_of_Loan'] = np.nan

# Num_of_Delayed_Payment: negative invalid
df.loc[df['Num_of_Delayed_Payment'] < 0, 'Num_of_Delayed_Payment'] = np.nan

# Monthly_Balance: extreme sentinel values like -3333333333333333333333333333
df.loc[df['Monthly_Balance'].abs() > 1e6, 'Monthly_Balance'] = np.nan

# Amount_invested_monthly: 10000 sentinel (__10000__) likely placeholder for
df.loc[df['Amount_invested_monthly'] == 10000, 'Amount_invested_monthly'] =

# --- categorical / text cleanup ---
def clean_cat(s, placeholders):
```

```
    if pd.isna(s):
        return np.nan
    s = str(s).strip()
    if s in placeholders:
        return np.nan
    return s

df['Occupation'] = df['Occupation'].apply(lambda x: clean_cat(x, {'_______'}
df['Credit_Mix'] = df['Credit_Mix'].apply(lambda x: clean_cat(x, {'_'}))
df['Payment_Behaviour'] = df['Payment_Behaviour'].apply(lambda x: clean_cat(
df['Name'] = df['Name'].apply(lambda x: clean_cat(x, {''}))

# Payment_of_Min_Amount: 'NM' = Not Mentioned -> treat as missing category (
df['Payment_of_Min_Amount'] = df['Payment_of_Min_Amount'].replace({'NM': np.

# --- Credit_History_Age: "22 Years and 1 Months" -> convert to total months
def parse_credit_history(s):
    if pd.isna(s):
        return np.nan
    s = str(s).strip()
    if s in ('NA', ''):
        return np.nan
    m = re.match(r'(\d+)\s*Years?\s*and\s*(\d+)\s*Months?', s)
    if m:
        years, months = int(m.group(1)), int(m.group(2))
        return years * 12 + months
    return np.nan

df['Credit_History_Age_Months'] = df['Credit_History_Age'].apply(parse_credi
df.drop(columns=['Credit_History_Age'], inplace=True)

# --- SSN: garbage values like '#F%$D@*&8' -> NaN ---
def clean_ssn(s):
    if pd.isna(s):
        return np.nan
    s = str(s).strip()
    if re.match(r'^\d{3}-\d{2}-\d{4}$', s):
        return s
    return np.nan
df['SSN'] = df['SSN'].apply(clean_ssn)

# --- Fill per-customer where possible (Name, Occupation, SSN, Age, Annual_I
#     are usually constant per Customer_ID across months) using group ffill/
group_stable_cols = ['Name', 'SSN', 'Occupation', 'Age', 'Annual_Income',
                     'Monthly_Inhand_Salary', 'Num_Bank_Accounts', 'Num_Cre
                     'Credit_History_Age_Months']
for c in group_stable_cols:
    df[c] = df.groupby('Customer_ID')[c].transform(lambda s: s.ffill().bfill

# --- Numeric type finalization ---
int_like = ['Num_Bank_Accounts', 'Num_Credit_Card', 'Interest_Rate', 'Delay_
for c in int_like:
    df[c] = pd.to_numeric(df[c], errors='coerce')

float_cols = ['Age','Annual_Income','Monthly_Inhand_Salary','Num_of_Loan','N
              'Changed_Credit_Limit','Num_Credit_Inquiries','Outstanding_Deb
```

```
              'Credit_Utilization_Ratio','Total_EMI_per_month','Amount_inves
              'Monthly_Balance','Credit_History_Age_Months']
for c in float_cols:
    df[c] = pd.to_numeric(df[c], errors='coerce')

# Type_of_Loan: strip whitespace
df['Type_of_Loan'] = df['Type_of_Loan'].astype(str).str.strip().replace({'na

# Credit_Score: standardize categories
df['Credit_Score'] = df['Credit_Score'].astype(str).str.strip()

# --- Remove exact duplicate rows ---
before_dedup = len(df)
df = df.drop_duplicates()
after_dedup = len(df)

# Reorder / save
df.to_csv(OUTPUT_PATH, index=False)

print("Original shape:", orig_shape)
print("Final shape:", df.shape)
print("Duplicates removed:", before_dedup - after_dedup)
print()
print("Missing values after cleaning:")
print(df.isna().sum())
print()
print(df.dtypes)

# --- Final imputation pass for remaining missing values ---
# Numeric: fill with median per Customer_ID group, fallback to global median
numeric_fill_cols = ['Num_of_Loan','Num_of_Delayed_Payment','Changed_Credit_
                     'Num_Credit_Inquiries','Amount_invested_monthly','Mont
for c in numeric_fill_cols:
    df[c] = df.groupby('Customer_ID')[c].transform(lambda s: s.fillna(s.medi
    df[c] = df[c].fillna(df[c].median())

# Categorical: fill with per-customer mode, fallback to 'Unknown'
def mode_or_nan(s):
    m = s.mode()
    return m.iloc[0] if not m.empty else np.nan

cat_fill_cols = ['Credit_Mix','Payment_of_Min_Amount','Payment_Behaviour']
for c in cat_fill_cols:
    df[c] = df.groupby('Customer_ID')[c].transform(lambda s: s.fillna(mode_c
    df[c] = df[c].fillna('Unknown')

# Type_of_Loan: missing likely means "no loan" (Num_of_Loan == 0)
df.loc[df['Type_of_Loan'].isna() & (df['Num_of_Loan'] == 0), 'Type_of_Loan']
df['Type_of_Loan'] = df['Type_of_Loan'].fillna('Unknown')

df.to_csv(OUTPUT_PATH, index=False)
print("=== FINAL ===")
print("Shape:", df.shape)
print(df.isna().sum().sum(), "total missing values remaining")
print(df.isna().sum()[df.isna().sum()>0])
```

```
Đã đọc file: /kaggle/input/datasets/holthin/credit/D3-CreditScore.csv
Shape ban đầu: (100000, 28)
       ID Customer_ID     Month           Name   Age          SSN Occupation
\
0  0x1602   CUS_0xd40   January  Aaron Maashoh    23  821-00-0265  Scientist
1  0x1603   CUS_0xd40  February  Aaron Maashoh    23  821-00-0265  Scientist
2  0x1604   CUS_0xd40     March  Aaron Maashoh  -500  821-00-0265  Scientist
3  0x1605   CUS_0xd40     April  Aaron Maashoh    23  821-00-0265  Scientist
4  0x1606   CUS_0xd40       May  Aaron Maashoh    23  821-00-0265  Scientist

  Annual_Income  Monthly_Inhand_Salary  Num_Bank_Accounts  ...  Credit_Mix
\
0      19114.12            1824.843333                  3  ...           _
1      19114.12                    NaN                  3  ...        Good
2      19114.12                    NaN                  3  ...        Good
3      19114.12                    NaN                  3  ...        Good
4      19114.12            1824.843333                  3  ...        Good

   Outstanding_Debt Credit_Utilization_Ratio     Credit_History_Age  \
0            809.98                26.822620  22 Years and 1 Months
1            809.98                31.944960                    NaN
2            809.98                28.609352  22 Years and 3 Months
3            809.98                31.377862  22 Years and 4 Months
4            809.98                24.797347  22 Years and 5 Months

   Payment_of_Min_Amount Total_EMI_per_month Amount_invested_monthly  \
0                     No           49.574949       80.41529543900253
1                     No           49.574949      118.28022162236736
2                     No           49.574949         81.699521264648
3                     No           49.574949       199.4580743910713
4                     No           49.574949      41.420153086217326

                 Payment_Behaviour     Monthly_Balance Credit_Score
0   High_spent_Small_value_payments  312.49408867943663         Good
1    Low_spent_Large_value_payments  284.62916249607184         Good
2   Low_spent_Medium_value_payments   331.2098628537912         Good
3    Low_spent_Small_value_payments  223.45130972736786         Good
4  High_spent_Medium_value_payments  341.48923103222177         Good

[5 rows x 28 columns]
Original shape: (100000, 28)
Final shape: (100000, 28)
Duplicates removed: 0

Missing values after cleaning:
ID                          0
Customer_ID                 0
Month                       0
Name                        0
Age                         0
SSN                         0
Occupation                  0
Annual_Income               0
Monthly_Inhand_Salary       0
Num_Bank_Accounts           0
Num_Credit_Card             0
```

```
Interest_Rate                   0
Num_of_Loan                  3876
Type_of_Loan                11408
Delay_from_due_date             0
Num_of_Delayed_Payment       7646
Changed_Credit_Limit         2091
Num_Credit_Inquiries         1965
Credit_Mix                  20195
Outstanding_Debt                0
Credit_Utilization_Ratio        0
Payment_of_Min_Amount       12007
Total_EMI_per_month             0
Amount_invested_monthly      8784
Payment_Behaviour            7600
Monthly_Balance              1209
Credit_Score                    0
Credit_History_Age_Months       0
dtype: int64

ID                           object
Customer_ID                  object
Month                        object
Name                         object
Age                         float64
SSN                          object
Occupation                   object
Annual_Income               float64
Monthly_Inhand_Salary       float64
Num_Bank_Accounts             int64
Num_Credit_Card               int64
Interest_Rate                 int64
Num_of_Loan                 float64
Type_of_Loan                 object
Delay_from_due_date           int64
Num_of_Delayed_Payment      float64
Changed_Credit_Limit        float64
Num_Credit_Inquiries        float64
Credit_Mix                   object
Outstanding_Debt            float64
Credit_Utilization_Ratio    float64
Payment_of_Min_Amount        object
Total_EMI_per_month         float64
Amount_invested_monthly     float64
Payment_Behaviour            object
Monthly_Balance             float64
Credit_Score                 object
Credit_History_Age_Months   float64
dtype: object
=== FINAL ===
Shape: (100000, 28)
0 total missing values remaining
Series([], dtype: int64)
```

1. Đọc dữ liệu vào bộ nhớ và hiển thị các thông số sau:

In [3]:
```
import pandas as pd
data = pd.read_csv("D3-CreditScore-cleaned.csv")
```

a. Kích thước và chiều của dữ liệu

- Dữ liệu có `100000` mẫu
- Dữ liệu có 28 chiều

In [4]:
```
data.shape
```

Out[4]:
```
(100000, 28)
```

b. Kiểu dữ liệu của các thuộc tính

In [5]:
```
data.columns
```

Out[5]:
```
Index(['ID', 'Customer_ID', 'Month', 'Name', 'Age', 'SSN', 'Occupation',
       'Annual_Income', 'Monthly_Inhand_Salary', 'Num_Bank_Accounts',
       'Num_Credit_Card', 'Interest_Rate', 'Num_of_Loan', 'Type_of_Loan',
       'Delay_from_due_date', 'Num_of_Delayed_Payment', 'Changed_Credit_Lim
it',
       'Num_Credit_Inquiries', 'Credit_Mix', 'Outstanding_Debt',
       'Credit_Utilization_Ratio', 'Payment_of_Min_Amount',
       'Total_EMI_per_month', 'Amount_invested_monthly', 'Payment_Behaviou
r',
       'Monthly_Balance', 'Credit_Score', 'Credit_History_Age_Months'],
      dtype='object')
```

In [6]:
```
data["Credit_Score"].unique()
```

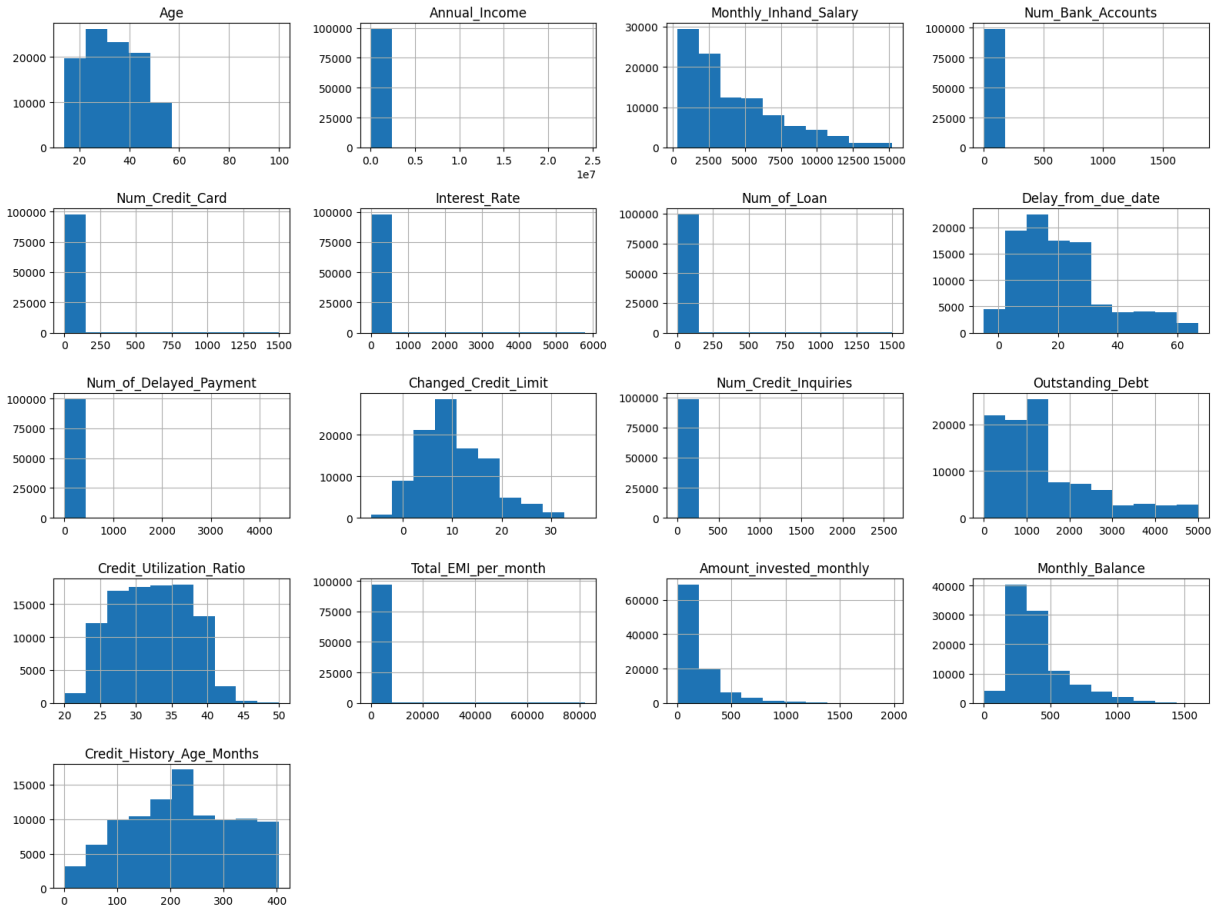
Out[6]:
```
array(['Good', 'Standard', 'Poor'], dtype=object)
```

c. Số lượng thực thể của các giá trị nhãn

- Dữ liệu có `credit_score` có `3` loại nhãng: `'Good', 'Standard', 'Poor'`
- và có `100000` mẫu

d. Giá trị lớn nhất, bé nhất, và trung bình của các cột thuộc tính có giá trị số thực

In [7]:
```
import matplotlib.pyplot as plt
data.hist(figsize=(16, 12))   # tăng kích thước tổng của hình
plt.tight_layout()
plt.show()
```



In [8]:
```
numeric_cols = data.select_dtypes(include=['number'])
summary = pd.DataFrame({
    'Max': numeric_cols.max(),
    'Min': numeric_cols.min(),
    'Mean': numeric_cols.mean()
})
```

In [9]:
```
summary
```

Out[9]:

	Max	Min	Mean
Age	1.000000e+02	14.000000	33.312380
Annual_Income	2.419806e+07	7005.930000	176415.701298
Monthly_Inhand_Salary	1.520463e+04	303.645417	4198.771619
Num_Bank_Accounts	1.798000e+03	-1.000000	17.091280
Num_Credit_Card	1.499000e+03	0.000000	22.474430
Interest_Rate	5.797000e+03	1.000000	72.466040
Num_of_Loan	1.496000e+03	0.000000	7.022880
Delay_from_due_date	6.700000e+01	-5.000000	21.068780
Num_of_Delayed_Payment	4.397000e+03	0.000000	29.709680
Changed_Credit_Limit	3.697000e+01	-6.490000	10.389113
Num_Credit_Inquiries	2.597000e+03	0.000000	27.321850
Outstanding_Debt	4.998070e+03	0.230000	1426.220376
Credit_Utilization_Ratio	5.000000e+01	20.000000	32.285173
Total_EMI_per_month	8.233100e+04	0.000000	1403.118217
Amount_invested_monthly	1.977326e+03	0.000000	193.686347
Monthly_Balance	1.602041e+03	0.007760	403.492393
Credit_History_Age_Months	4.040000e+02	1.000000	221.136220

2. Tiền xử lý dữ liệu và trực quan hóa

a. Kiểm tra các thuộc tính của dữ liệu, xem xét bỏ những cột không có ý nghĩa dự đoán (như tên,...). Giải thích tại sao các cột này cần xóa.

Các loại dữ liệu nên bỏ

Feature	Lí do
ID	Chỉ là mã định danh dòng, không mang thông tin gì để dự đoán
Customer_ID	Định danh khách hàng, không phải đặc trưng tài chính

Feature	Lí do
Name	Tên riêng, không liên quan đến điểm tín dụng
SSN	Số định danh cá nhân, không có ý nghĩa dự đoán, còn rủi ro về dữ liệu nhạy cảm
Month	Thường không ảnh hưởng nhiều đến điểm tín dụng (trừ khi bạn phân tích xu hướng theo thời gian)
Type_of_Loan	Dạng text tự do, nhiều loại kết hợp phức tạp (vd "Auto Loan, Credit-Builder..."), khó dùng trực tiếp — nếu muốn giữ thì phải tách thành các cột nhị phân (one-hot) cho từng loại vay

Các loại dữ liệu nên giữ lại

Nhóm	Cột
Thông tin độ tuổi	Age, Occupation
Thu nhập	Annual_Income, Monthly_Inhand_Salary
Tài khoản/thẻ	Num_Bank_Accounts, Num_Credit_Card, Interest_Rate
Vay nợ	Num_of_Loan, Outstanding_Debt, Total_EMI_per_month
Lịch sử thanh toán	Delay_from_due_date, Num_of_Delayed_Payment, Payment_of_Min_Amount, Payment_Behaviour
Tín dụng	Changed_Credit_Limit, Num_Credit_Inquiries, Credit_Mix, Credit_Utilization_Ratio, Credit_History_Age_Months
Tài chính khác	Amount_invested_monthly, Monthly_Balance

b. Tìm giải pháp chuyển hết các thuộc tính về dạng số. Mô tả rõ giải pháp làm.

- Chuyển các dữ liệu về dạng số để có thể dựa vào đó đánh gá các dữ liệu dạng `categorical`

In [10]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

def prepare_features(df):
    """
    Giữ lại các feature cần thiết cho việc dự đoán Credit_Score
    và encode các cột categorical thành dạng số.
```

```
    Input:
        df: DataFrame đã được làm sạch (D3-CreditScore-cleaned.csv)

    Output:
        X: DataFrame chỉ chứa các feature đã encode (sẵn sàng đưa vào model)
        y: Series chứa biến mục tiêu Credit_Score (dạng số)
        encoders: dict lưu lại các LabelEncoder đã dùng (để decode lại sau n
    """

    # ===== 1. Chọn các cột cần giữ =====
    keep_cols = [
        'Age', 'Occupation',
        'Annual_Income', 'Monthly_Inhand_Salary',
        'Num_Bank_Accounts', 'Num_Credit_Card', 'Interest_Rate',
        'Num_of_Loan', 'Outstanding_Debt', 'Total_EMI_per_month',
        'Delay_from_due_date', 'Num_of_Delayed_Payment',
        'Payment_of_Min_Amount', 'Payment_Behaviour',
        'Changed_Credit_Limit', 'Num_Credit_Inquiries', 'Credit_Mix',
        'Credit_Utilization_Ratio', 'Credit_History_Age_Months',
        'Amount_invested_monthly', 'Monthly_Balance',
        'Credit_Score'  # target - sẽ tách riêng ra sau
    ]
    df = df[keep_cols].copy()

    # ===== 2. Encode các cột categorical =====
    encoders = {}

    # --- Occupation: nhiều nhóm không có thứ tự -> Label Encoding (hoặc on
    le_occ = LabelEncoder()
    df['Occupation'] = le_occ.fit_transform(df['Occupation'].astype(str))
    encoders['Occupation'] = le_occ

    # --- Payment_of_Min_Amount: Yes/No/Unknown -> map thủ công (có ý nghĩa
    payment_map = {'No': 0, 'Yes': 1, 'Unknown': -1}
    df['Payment_of_Min_Amount'] = df['Payment_of_Min_Amount'].map(payment_ma

    # --- Payment_Behaviour: không có thứ tự rõ ràng -> Label Encoding ---
    le_pb = LabelEncoder()
    df['Payment_Behaviour'] = le_pb.fit_transform(df['Payment_Behaviour'].as
    encoders['Payment_Behaviour'] = le_pb

    # --- Credit_Mix: CÓ thứ tự tự nhiên (Bad < Standard < Good) -> map thủ
    credit_mix_map = {'Bad': 0, 'Standard': 1, 'Good': 2, 'Unknown': -1}
    df['Credit_Mix'] = df['Credit_Mix'].map(credit_mix_map).fillna(-1).astyp

    # --- Credit_Score (target): CÓ thứ tự tự nhiên (Poor < Standard < Good)
    score_map = {'Poor': 0, 'Standard': 1, 'Good': 2}
    df['Credit_Score'] = df['Credit_Score'].map(score_map)

    # ===== 3. Tách X và y =====
    y = df['Credit_Score']
    X = df.drop(columns=['Credit_Score'])

    return X, y, encoders
```

- Occupation: nhiều nhóm không có thứ tự -> Label Encoding
- Payment_of_Min_Amount: Yes/No/Unknown -> map thủ công (có ý nghĩa thứ tự nhị phân)
- Payment_Behaviour: không có thứ tự rõ ràng -> Label Encoding
- Credit_Mix: CÓ thứ tự tự nhiên (Bad < Standard < Good) -> map thủ công theo thứ tự
- Credit_Score (target): CÓ thứ tự tự nhiên (Poor < Standard < Good)

In [11]:
```
X, y, encoders = prepare_features(data)

print("Shape X:", X.shape)
print("Shape y:", y.shape)
print()
print(X.dtypes)
print()
print(X.head())
print()
print("Phân bố´target y:")
print(y.value_counts())
```

```
Shape X: (100000, 21)
Shape y: (100000,)

Age                         float64
Occupation                    int64
Annual_Income               float64
Monthly_Inhand_Salary       float64
Num_Bank_Accounts             int64
Num_Credit_Card               int64
Interest_Rate                 int64
Num_of_Loan                 float64
Outstanding_Debt            float64
Total_EMI_per_month         float64
Delay_from_due_date           int64
Num_of_Delayed_Payment      float64
Payment_of_Min_Amount         int64
Payment_Behaviour             int64
Changed_Credit_Limit        float64
Num_Credit_Inquiries        float64
Credit_Mix                    int64
Credit_Utilization_Ratio    float64
Credit_History_Age_Months   float64
Amount_invested_monthly     float64
Monthly_Balance             float64
dtype: object

    Age  Occupation  Annual_Income  Monthly_Inhand_Salary  Num_Bank_Accounts
\
0  23.0          12       19114.12            1824.843333                  3
1  23.0          12       19114.12            1824.843333                  3
2  23.0          12       19114.12            1824.843333                  3
3  23.0          12       19114.12            1824.843333                  3
4  23.0          12       19114.12            1824.843333                  3

   Num_Credit_Card  Interest_Rate  Num_of_Loan  Outstanding_Debt  \
0                4              3          4.0            809.98
1                4              3          4.0            809.98
2                4              3          4.0            809.98
3                4              3          4.0            809.98
4                4              3          4.0            809.98

   Total_EMI_per_month  ...  Num_of_Delayed_Payment  Payment_of_Min_Amount
\
0            49.574949  ...                     7.0                      0
1            49.574949  ...                     6.5                      0
2            49.574949  ...                     7.0                      0
3            49.574949  ...                     4.0                      0
4            49.574949  ...                     6.5                      0

   Payment_Behaviour  Changed_Credit_Limit  Num_Credit_Inquiries  Credit_Mix
\
0                  2                 11.27                   4.0           2
1                  3                 11.27                   4.0           2
2                  4                 11.27                   4.0           2
3                  5                  6.27                   4.0           2
4                  1                 11.27                   4.0           2
```

```
   Credit_Utilization_Ratio  Credit_History_Age_Months  \
0                 26.822620                      265.0
1                 31.944960                      265.0
2                 28.609352                      267.0
3                 31.377862                      268.0
4                 24.797347                      269.0

   Amount_invested_monthly  Monthly_Balance
0                80.415295       312.494089
1               118.280222       284.629162
2                81.699521       331.209863
3               199.458074       223.451310
4                41.420153       341.489231

[5 rows x 21 columns]

Phân bố target y:
Credit_Score
1    53174
0    28998
2    17828
Name: count, dtype: int64
```

In [12]:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# ===== 2. Chuẩn hóa dữ liệu (PCA rất nhạy với scale) =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ===== 3. Giảm chiều xuống 2D bằng PCA =====
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

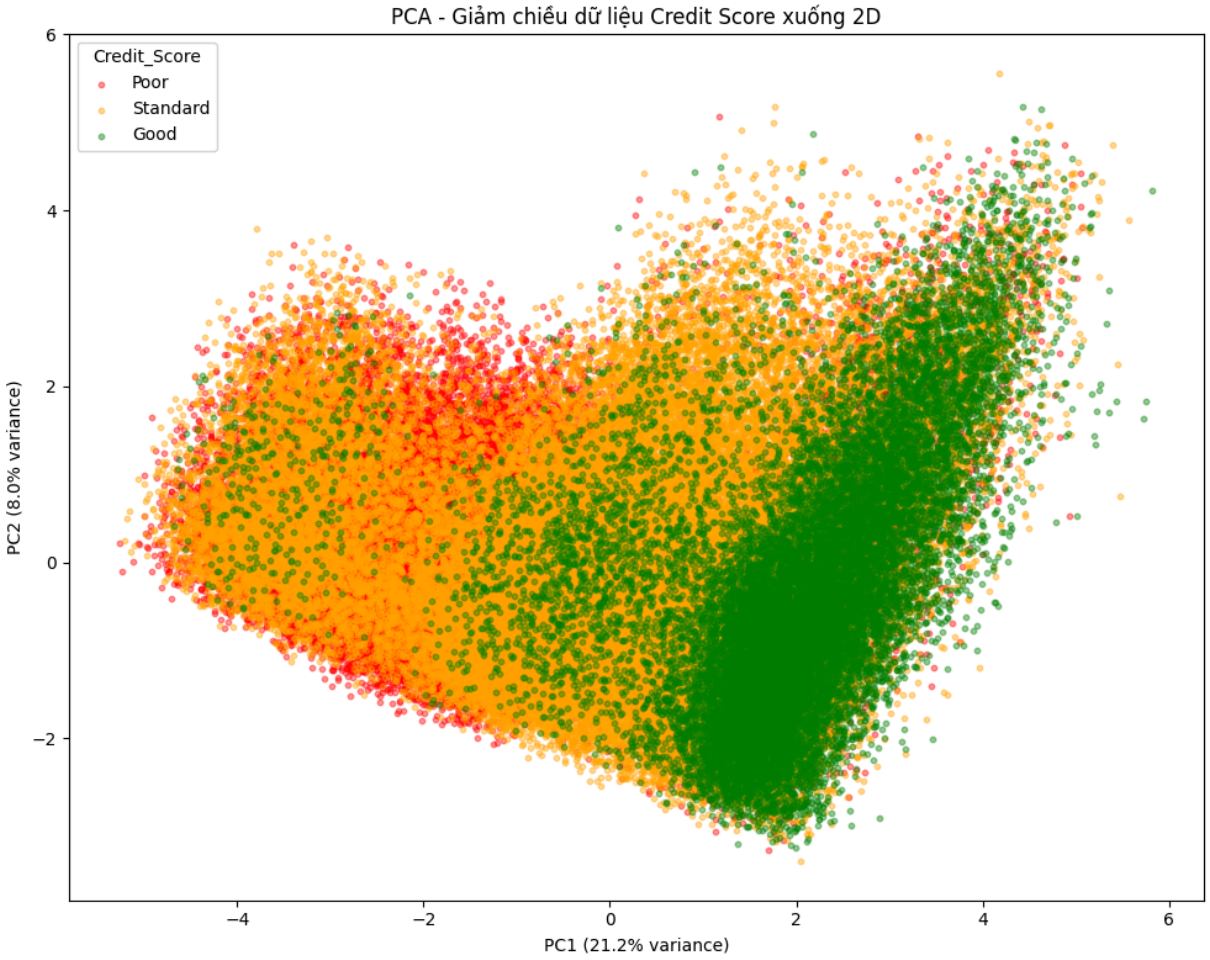
print("Tỉ lệ phương sai giữ được của 2 thành phần:", pca.explained_variance_
print("Tổng phương sai giữ được:", pca.explained_variance_ratio_.sum())

# ===== 4. Vẽ scatter plot 2D, mỗi nhãn 1 màu =====
label_names = {0: 'Poor', 1: 'Standard', 2: 'Good'}
colors = {0: 'red', 1: 'orange', 2: 'green'}

plt.figure(figsize=(10, 8))
for label_value, label_name in label_names.items():
    mask = (y == label_value)
    plt.scatter(
        X_pca[mask, 0], X_pca[mask, 1],
        c=colors[label_value],
        label=label_name,
        alpha=0.4,
        s=10
    )
```

```
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.title("PCA - Giảm chiều dữ liệu Credit Score xuống 2D")
plt.legend(title="Credit_Score")
plt.tight_layout()
plt.savefig("pca_2d.png", dpi=150)
plt.show()
print("Đã lưu hình vào pca_2d.png")
```

```
Tỉ lệ phương sai giữ được của 2 thành phần: [0.2118008  0.08018848]
Tổng phương sai giữ được: 0.2919892802496876
```




```
Đã lưu hình vào pca_2d.png
```

In [19]:

```
import pandas as pd
import numpy as np
import time
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import (StratifiedKFold, cross_validate,
                                     GridSearchCV, train_test_split)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

RANDOM_STATE = 42

# ===== 1. Load & chuẩn bị dữ liệu =====
```

```
X_sub, _, y_sub, _ = train_test_split(
    X, y, train_size=1000, stratify=y, random_state=RANDOM_STATE
)
print("Kích thước tập dùng để huấn luyện/so sánh:", X_sub.shape)
print("Phân bố nhãn:\n", y_sub.value_counts(normalize=True))

# ===== 2. Chuẩn hóa dữ liệu (cần cho KNN, SVM; không hại RF/AdaBoost) ===
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_sub)

# ===== 3. Chọn 3 mô hình: Random Forest, AdaBoost, KNN =====
# (Bỏ SVM vì quá chậm với dữ liệu lớn - O(n^2) đến O(n^3); Naive Bayes cho
#  do giả định các feature độc lập, không phù hợp với dữ liệu tài chính có

param_grids = {
    "KNN": {
        "estimator": KNeighborsClassifier(),
        "params": {
            "n_neighbors": [5, 11, 21],
            "weights": ["uniform", "distance"],
        }
    },
    "Random Forest": {
        "estimator": RandomForestClassifier(random_state=RANDOM_STATE, n_job
        "params": {
            "n_estimators": [100, 200],
            "max_depth": [10, 20, None],
            "min_samples_split": [2, 5],
        }
    },
    "AdaBoost": {
        "estimator": AdaBoostClassifier(random_state=RANDOM_STATE),
        "params": {
            "n_estimators": [50, 100, 200],
            "learning_rate": [0.1, 0.5, 1.0],
        }
    },
}

best_models = {}
tuning_report = {}

for name, cfg in param_grids.items():
    t0 = time.time()
    grid = GridSearchCV(
        cfg["estimator"], cfg["params"],
        cv=5, scoring="f1_macro", n_jobs=1
    )
    grid.fit(X_scaled, y_sub)
    elapsed = time.time() - t0
    best_models[name] = grid.best_estimator_
    tuning_report[name] = {
        "best_params": grid.best_params_,
        "best_cv_f1_macro": grid.best_score_,
        "tuning_time_sec": round(elapsed, 1)
```

```
    }
    print(f"[{name}] best_params={grid.best_params_}  "
          f"f1_macro={grid.best_score_:.4f}  time={elapsed:.1f}s")
```

```
Kích thước tập dùng để huấn luyện/so sánh: (1000, 21)
Phân bố nhãn:
 Credit_Score
1    0.532
0    0.290
2    0.178
Name: proportion, dtype: float64
[KNN] best_params={'n_neighbors': 21, 'weights': 'distance'}  f1_macro=0.587
8  time=0.2s
[Random Forest] best_params={'max_depth': None, 'min_samples_split': 2, 'n_e
stimators': 200}  f1_macro=0.6469  time=31.2s
[AdaBoost] best_params={'learning_rate': 0.5, 'n_estimators': 100}  f1_macro
=0.6283  time=19.9s
```

- Random Forest cho F1-macro cao nhất (0.647) — mô hình mạnh nhất trong 3 mô hình ở bước tune này, vì Random Forest xử lý tốt dữ liệu có tương tác phức tạp giữa các feature (thu nhập, nợ, lịch sử tín dụng...) mà không cần giả định tuyến tính.
- AdaBoost đứng thứ 2 (0.628) — khá gần Random Forest, cũng là mô hình ensemble dựa trên boosting cây quyết định yếu.
- KNN thấp nhất (0.588) — vì KNN dựa hoàn toàn vào khoảng cách Euclidean giữa các điểm, dễ bị ảnh hưởng bởi "curse of dimensionality" khi có 20 feature, không nắm bắt tốt các mối quan hệ phi tuyến phức tạp như 2 mô hình cây kia. Kết quả này thấp hơn một chút so với lần chạy với mẫu 10,000 dòng trước đó (RF đạt 0.703) — vì mẫu 1,000 dòng nhỏ hơn nên mô hình học được ít thông tin hơn, dễ bị nhiễu/phương sai cao hơn. Nếu máy bạn đủ nhanh, nên dùng mẫu lớn hơn (5,000-10,000+) để kết quả ổn định và đáng tin cậy hơn.

- ARI ≈ 0.09 (K-Means) — rất thấp, gần với mức "gom cụm ngẫu nhiên" (ARI=0 nghĩa là ngẫu nhiên). Cụm K-Means tìm ra chỉ hơi tốt hơn đoán bừa một chút.
- DBSCAN gần như 0 ở mọi chỉ số — vì nó chỉ tạo ra 1 cụm lớn, nên về mặt thống kê nó gần như "không phân loại được gì", khớp ngẫu nhiên với nhãn thật.
- Kết luận: K-Means vẫn nhỉnh hơn DBSCAN, nhưng cả hai đều cho thấy cấu trúc cụm tự nhiên của dữ liệu không khớp tốt với 3 nhãn Credit_Score — đúng như đã bàn ở câu trước, dữ liệu này các nhóm chồng lấn nhiều trong không gian đặc trưng, nên gom cụm không giám sát không tách rõ được.

In [23]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import (adjusted_rand_score, normalized_mutual_info_sco
                             homogeneity_score, completeness_score, v_measu
```

```
                                silhouette_score)

X_sample, _, y_sample, _ = train_test_split(
    X, y, train_size=1000, stratify=y, random_state=RANDOM_STATE
)

# ===== 3. Chuẩn hóa dữ liệu (bắt buộc với K-Means/DBSCAN vì dựa trên khoa
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_sample)

# ===== 4. Gom cụm bằng K-Means (k=3 vì có 3 nhãn: Poor/Standard/Good) =====
kmeans = KMeans(n_clusters=3, random_state=RANDOM_STATE, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)

# ===== 5. Gom cụm bằng DBSCAN =====
# DBSCAN không cần chọn số cụm trước, nhưng cần tune eps và min_samples.
# Dùng k-distance graph (k=min_samples) để ước lượng eps hợp lý thay vì đoá
from sklearn.neighbors import NearestNeighbors
k = 10
nn = NearestNeighbors(n_neighbors=k).fit(X_scaled)
distances, _ = nn.kneighbors(X_scaled)
k_distances = np.sort(distances[:, -1])
eps_estimate = np.percentile(k_distances, 90)   # điểm "khuỷu tay" xấp xỉ
print(f"eps ước lượng từ k-distance graph: {eps_estimate:.3f}")

dbscan = DBSCAN(eps=eps_estimate, min_samples=k)
dbscan_labels = dbscan.fit_predict(X_scaled)   # nhãn -1 = điểm nhiễu (noi

n_clusters_dbscan = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else
n_noise = list(dbscan_labels).count(-1)
print(f"DBSCAN tìm được {n_clusters_dbscan} cụm, {n_noise} điểm nhiễu ({n_no

# ===== 6. Đánh giá kết quả gom cụm bằng nhãn thật (external validation) =
def evaluate_clustering(y_true, cluster_labels, name):
    ari = adjusted_rand_score(y_true, cluster_labels)
    nmi = normalized_mutual_info_score(y_true, cluster_labels)
    homogeneity = homogeneity_score(y_true, cluster_labels)
    completeness = completeness_score(y_true, cluster_labels)
    v_measure = v_measure_score(y_true, cluster_labels)
    print(f"\n=== {name} ===")
    print(f"Adjusted Rand Index (ARI):      {ari:.4f}")
    print(f"Normalized Mutual Info (NMI):   {nmi:.4f}")
    print(f"Homogeneity:                    {homogeneity:.4f}")
    print(f"Completeness:                   {completeness:.4f}")
    print(f"V-measure:                      {v_measure:.4f}")
    return {"Model": name, "ARI": ari, "NMI": nmi,
            "Homogeneity": homogeneity, "Completeness": completeness,
            "V-measure": v_measure}

results = []
results.append(evaluate_clustering(y_sample, kmeans_labels, "K-Means (k=3)")
results.append(evaluate_clustering(y_sample, dbscan_labels, "DBSCAN"))

# ===== 7. Bảng so sánh cuối cùng =====
results_df = pd.DataFrame(results)
print("\n" + "="*80)
```

```
print(results_df.to_string(index=False))
results_df.to_csv("clustering_comparison.csv", index=False)
```

```
eps ước lượng từ k-distance graph: 3.726
DBSCAN tìm được 1 cụm, 88 điểm nhiễu (8.8%)

=== K-Means (k=3) ===
Adjusted Rand Index (ARI):      0.0892
Normalized Mutual Info (NMI):   0.1148
Homogeneity:                    0.1187
Completeness:                   0.1111
V-measure:                      0.1148

=== DBSCAN ===
Adjusted Rand Index (ARI):      0.0088
Normalized Mutual Info (NMI):   0.0017
Homogeneity:                    0.0011
Completeness:                   0.0038
V-measure:                      0.0017

================================================================================
====
        Model      ARI      NMI  Homogeneity  Completeness  V-measure
K-Means (k=3) 0.089198 0.114756     0.118703      0.111064   0.114756
       DBSCAN 0.008798 0.001745     0.001132      0.003808   0.001745
```

In [ ]: